

sudo usermod -aG - adauga utilizator in grup -G → specifică lista de grupuri secundare -a (append) → adaugă grupul fără să șteargă cele existente usermod -l nou nume # redenumeste user usermod -L nume # blochează cont usermod -U nume # deblochează

cat /etc/passwd - afiseaza toti utilizatori id nume # UID + grupuri groups nume # grupuri whoami # user curent who # cine e logat cat /etc/group - afiseaza toate grupurile getent group gods- vizualizare membrii unui grup

/etc/passwd → utilizatori /etc/group → grupuri id / groups → relația user grupuri

sudo passwd -l adonis - blocheaza parola lui adonis passwd -l nume # blochează contul (lock) passwd -u nume # deblochează passwd -d nume # șterge parola passwd -S nume # status parolă chage -M 90 nume # parola expiră după 90 zile chage -m 7 nume # minim 7 zile între schimbări chage -W 7 nume # avertizare înainte de expirare chage -E YYYY-MM-DD nume # expirare cont chage -l nume # afișare info

\$? codul de ieșire al ultimei comenzi \$\$ PID-ul (procesul) scriptului curent

name="Ana" echo \$name name="Ana" → creăm o variabilă numită name cu valoarea "Ana" \$name → înlocuiește cu valoarea variabilei → aici va afișa Ana

Fără \$:

echo name afișează literal name, nu valoarea ei

dd este o comandă de nivel jos pentru copiere și conversie a datelor Lucrează bloc cu bloc (nu linie cu linie) Este folosită pentru: Crearea de fișiere mari goale sau pline cu zerouri Scrierea directă pe dispozitive (USB, hard disk) Backup de partiții sau imagini ISO Conversie între formate (ASCII → EBCDIC etc.) 2.Sintaxă generală dd if=<input> of=<output> [opțiuni] Parametru Ce înseamnă if= input file – sursa datelor of= output file – fișierul/ dispozitivul destinație bs= block size – dimensiunea unui bloc (ex: 1M = 1MB) count= numărul de blocuri de copiat status= ce mesaj să afișeze (none, progress, noxfer)

*Schimbarea shell ului sudo chsh -s \$(which zsh) lambda chsh -s → schimbă shell-ul \$(which zsh) → calea completă către zsh (de obicei /usr/bin/zsh)

wc: -l Afișează doar numărul de linii -w Afișează doar numărul de cuvinte -c Afișează doar numărul de bytes -m Afișează doar numărul de caractere (diferență față de bytes dacă textul are Unicode) -L Afișează lungimea liniei celei mai lungi ^ = începutul liniei \$ = sfârșitul liniei

awk = analiză și procesare pe coloane, cu calcul și filtrare sed = modificare și filtrare a textului linie cu linie

fork() = creează un proces copil pornind de la procesul părinte Procesul copil este o copie aproape identică a părintelui: Are același cod, date, stack Are un PID diferit

exec() = înlocuiește imaginea procesului curent cu un nou program După exec, procesul curent nu mai există în forma inițială; codul și datele părintelui sunt înlocuite

Creează arhivă .tar (fără compresie) tar -cvf arhiva.tar folder/ c = create v = verbose (afișează ce face) f = file Cu compresie gzip (.tar.gz) tar -czvf arhiva.tar.gz folder/ z = gzip Cu compresie bzip2 (.tar.bz2) tar -cjvf arhiva.tar.bz2 folder/ j = bzip2 Extrage arhivă tar -xvf arhiva.tar tar -xzvf arhiva.tar.gz x = extract 2. zip zip = arhivare + compresie (face ambele direct) Format mai portabil (Windows, etc.) Exemple Creează arhivă zip zip -r arhiva.zip folder/ -r = recursive (include subfoldere) Extrage arhivă zip unzip arhiva.zip

fallocate -l(lenght/dimensiune) - da unui fisier un anumit spatiu, ocupa spatiu pe disc pentru acel fisier

gcc <nume fisier cu codul sursa> -o <nume executabil>

Compilarea si rularea unui program din mai multe surse: 1 student@uso:~\$ gcc -c utils.c 2 student@uso:~\$ gcc -c main.c 3 student@uso:~\$ gcc utils.o main.o -o main 4 student@uso:~\$./main

optiuni gcc:

Bază -o fisier Setează numele executabilului -c Compilează fără linking (.o) (transforma limnajul in cod masina dar nu se creeaza programul executabil) -S Generează cod assembly -E Rulează doar preprocesorul Optimizare -O0 Fără optimizare -O1 Optimizare de bază -O2 Optimizare recomandată -O3 Optimizare maximă -Os Optimizează dimensiunea Debug -g Include informații pentru debugging Warning-uri -Wall Activează warning-uri uzuale -Wextra Warning-uri suplimentare -Werror Tratează warning-urile ca erori Biblioteci -I<dir> Adaugă director pentru header files -L<dir> Adaugă director pentru librării -l<nume> Leagă o bibliotecă (ex: -lm)

Tip fișier Ce este .c cod sursă (NU executabil) .o cod compilat (intermediar) program / .exe executabil (rulează)

| - trimite iesirea unui comenzi ca intrare pentru alta comanda &&(si logic) - a doua comanda se executa doar daca prima reuseste ||(sau logic) - a doua comanda se executa doar daca prima a esuat & - trimite procesul in background

unset - anulara declararii unei variabile(stergea valorii) export -p / env - afiseaza toate variabilele de mediu(o variabila de mediu este o variabila care este mostenita de procesele create de shell, de ex: USER,HOME,TERM,... . Folosim variabila de mediu atunci cand vrem sa fie mostenita si de alte procese, nu sa afecteze doar shell ul curent)

Pentru a defini o variabila de mediu vom folosi tot comanda export(export <nume variabila> o va transforma in variabila de mediu si cand vom folosi omanda env va aparea ca variabila de mediu)

export -n - face sa nu mai fie variabila de mediu dar ramane ca variabila definita(declarata)

$\$(\dots)$ - shell ul calculeaza(ex: $\$(3+4)$ - afiseaza 7)

Expandare - shell ul interpreteaza simbolurile(ex: ~ ce inseamna home,adica inlocuie cu valoarea variabilei HOME, test{1,2,3} - shell ul va afisa test1, test2, test3

Atenție la ambiguități echo \$config__test

Shell-ul caută variabila config__test

Dar dacă vrei:

echo \${config}__test

Se întâmplă:

$\${config}$ → /etc/nsswitch.conf apoi se adaugă __test

Rezultat:

/etc/nsswitch.conf__test

~ → directorul home {} → generează variante multiple \$VAR → valoarea unei variabile \${VAR} → formă clară (evită confuzii) $\$(\)$ → calcule matematice

Globbering (sau pathname expansion) înseamnă că shell-ul înlocuiește anumite pattern-uri (șabloane) cu nume reale de fișiere/directoare.

Practic: nu scrii numele exact al fișierelor → scrii un tipar → shell-ul găsește toate potrivirile

Exemple din text 1. Fișiere care încep cu „z” ls /bin/z*

z* înseamnă:

„toate numele care încep cu z”

Devine ceva de genul:

/bin/zcat /bin/zcmp /bin/zdiff ... 2. Fișiere care se termină cu „w” ls /bin/*w

*w înseamnă:

„toate numele care se termină în w”

Exemplu rezultat:

/bin/znew Cele mai importante simboluri de globbering * (asterisc)

înlocuiește orice șir de caractere

*.txt

→ toate fișierele .txt

?

înlocuiește un singur caracter

file?.txt

→ file1.txt, fileA.txt etc.

[abc]

unul dintre caracterele specificate

file[12].txt

→ file1.txt, file2.txt

[a-z]

interval de caractere

file[a-z].txt

? – se potrivește cu exact un singur caracter * – se potrivește cu oricâte caractere (inclusiv zero) [...] – se potrivește cu oricare caracter din mulțimea specificată între paranteze

Moduri de escaping: / - backslash " " - ghilimele ' ' - apostrofuri

Configurari si personalizarea: -global, la nivelul sistemului: /etc/bash.bashrc
-local, la nivelul utilizatorului: ~/ .bashrc

Promptul este textul pe care îl vezi în terminal înainte să scrii comenzi, de exemplu:

student@uso:~\$

El este controlat de variabila:

PS1 2. Ce este PS1

PS1 = „Primary Shell Prompt”

Este o variabilă care spune shell-ului cum să afișeze linia de comandă.

Ex:

echo \$PS1

îți arată „rețeta” promptului.

3. Ce conține PS1

PS1 nu este doar text simplu, ci include coduri speciale interpretate de shell:

Exemple importante (din tabel) \u → numele utilizatorului \h → numele calculatorului (hostname) \w → directorul curent \s → \$ (sau # dacă e root) \t → ora (format 24h) \@ → ora în format 12h (AM/PM) \[\033[...m\] → culori în terminal 4. Exemple din text Prompt standard \u@\h:\w\$

devine:

student@uso:~\$

\u → numele utilizatorului

\h → numele stației (hostname)

\w → directorul curent

\\$ → caracterul \$ (sau # pentru root)

\t → ora curentă în format 24h

\@ → ora curentă în format 12h (AM/PM)

\[\033[COLORm\] → schimbă culoarea textului din prompt

comanda: stat -c "%n,%s" /etc/* | sort -n -k 2 -t ' ' | tail -5

semnul % apare în argumentul pentru stat și are un rol special.

Ce înseamnă % în stat -c

Opțiunea -c (format) din stat funcționează ca un șablon de afișare. În acest șablon, secvențele care încep cu % sunt placeholder (coduri speciale) care se înlocuiesc cu informații despre fișier.

% marchează variabile speciale în formatul stat, care sunt înlocuite cu informații reale despre fișier.

În cazul tău: "%n,%s" %n → numele fișierului %s → dimensiunea fișierului (în bytes)

Deci fiecare fișier din /etc/ va fi afișat ca:

nume_fisier,dimensiune

Exemplu:

/etc/mime.types,24301 Restul comenzii (pe scurt) stat -c "%n,%s" /etc/* → listează toate fișierele cu nume + dimensiune sort -n -k 2 -t ' ' → sortează numeric după coloana 2 (dimensiunea), folosind , ca separator tail -5 → ia ultimele 5 (adică cele mai mari fișiere)

-sed sed 's/al/AL/g' < test.txt Explicație: s = substitute (înlocuiește) al = textul căutat AL = textul nou g = global (toate aparițiile pe fiecare linie)

Înlocuiește toate aparițiile „al” cu „AL”

sed -n '/ana/,/si/p' < test.txt Explicație: -n = nu afișă automat toate liniile /ana/,/si/ = interval (de la linia care conține „ana” până la prima care conține „si”) p = print (afișează)

Afișează doar liniile între „ana” și primul „si” inclusiv

sed '/si/d' < test.txt Explicație: /si/ = selectează liniile care conțin „si” d = delete (șterge)

Elimină toate liniile care conțin „si” Tipuri de operații frecvente cu sed:

s/.../.../ → înlocuire /pattern/p → afișare selectivă /pattern/d → ștergere intervale /a/,/b/

FIND

1–5: Căutări simple find uso.git/

Listează tot din director (recursiv: fișiere + directoare)

find uso.git/ -type f

Afișează doar fișierele (f = file)

find uso.git/ -type f -name 'a*'

Găsește fișiere:

care sunt fișiere (-type f) al căror nume începe cu a find uso.git/ -type f -perm -111

Găsește fișiere care au permisiune de execuție pentru:

user grup alții

-111 = toate cele 3 au bit de execuție

find uso.git/ -type f -mtime -25

Găsește fișiere modificate în ultimele 25 de zile

-25 = mai recent de 25 zile +25 = mai vechi de 25 zile 7: Ștergere fișiere find uso.git/ -type f -perm -111 -delete

Șterge toate fișierele executabile

Atenție:

nu cere confirmare este periculos dacă nu ești sigur 8: Execuție comandă pe rezultate find uso.git/ -type f -perm -111 -exec ls -l {} \;

Pentru fiecare fișier găsit:

rulează ls -l Explicație: -exec = execută comandă {} = înlocuit cu numele fișierului \; = terminatorul comenzii 9–11: Folosire stat find uso.git/ -type f -exec stat -c "%s" {} \;

Afișează dimensiunea fiecărui fișier

find uso.git/ -type f -exec stat -c "%s,%n" {} \;

Afișează:

dimensiune,nume find uso.git/ -type f -exec stat -c "%s,%n" {} \; | sort -n

Sortează fișierele după dimensiune (numeric)

13: Alternativă cu xargs find uso.git/ -type f -name 'a*' | xargs ls -l

Echivalent cu:

-exec ls -l {} \; Diferența: xargs grupează argumentele → mai eficient -exec rulează comanda pentru fiecare fișier Ideea generală

find = caută

filtre: -type -name -perm -mtime acțiuni: -delete -exec | xargs

Simboluri Globbing Simbol Semnificație Exemplu * orice număr de caractere *.txt ? exact un caracter file?.txt [] un caracter din set file[123].txt [a-z] interval de caractere file[a-z].txt [!abc] orice caracter EXCEPTAT file[!123].txt {} expansion listă (brace expansion) file{1,2,3}.txt ~ home directory cd ~ ** recursiv (cu globstar activat) **/*.txt Exemple Globbing *.log

toate fișierele .log

file?.txt

file1.txt, fileA.txt

file[1-5].txt

file1.txt până la file5.txt

file{a,b,c}.txt

expandează în:

filea.txt fileb.txt filec.txt 2. REGEX (Regular Expressions)

Regex este folosit pentru:

grep sed awk perl vim programare

Exemplu:

grep "^root" /etc/passwd 3. Simboluri Regex Simbol Semnificație Exemplu . orice caracter a.c ^ început de linie ^root \$ sfârșit de linie txt\$ * 0 sau mai multe ab*c + 1 sau mai multe ab+c ? 0 sau 1 colou?r [] set de caractere [abc] [^] negare set [^0-9] [a-z] interval [a-z] () grupare (abc) | OR cat|dog {n} exact n apariții a{3} {n,} minim n a{2,} {n,m} între n și m a{2,5} \ escape \. \b boundary cuvânt \bcat\b \d cifră (unele regex) \d+ \w literă/cifră/_ \w+ \s spațiu alb \s+ 4. Diferența IMPORTANTĂ Globbing *.txt

lucrează cu nume de fișiere

Regex .*\.txt\$

lucrează cu text

{a,b,c} sau {1..20} etc. nu este nici globbing nici regex, este brace expansion

[a-zA-Z0-9._%+-] - aici . , + sau alte caractere regex nu mai sunt interpretate de exemplu . nu mai este interpretat orice caracter deoarece se afla între []

comanda "uname" - afiseaza numele kernel ului(ex: linux), pt detalii uname -a

Utilitarele folosite pentru administrarea partițiilor diferă în funcție de sistemul de operare:

Pe Linux se pot utiliza:

`fdisk` – funcționează doar cu schema MBR `gdisk` – destinat discurilor cu GPT
`parted` și `gparted` – oferă suport atât pentru MBR, cât și pentru GPT, precum și funcții avansate de gestionare

După crearea unei partiții, aceasta apare în directorul `/dev` și este denumită folosind numele discului (de exemplu `sda`), urmat de numărul partiției (de exemplu `sda5`).

Pe Windows, administrarea partițiilor se face prin utilitarul Disk Management. După creare, partițiile pot fi vizualizate și gestionate direct în această interfață.

comanda `blkid` - pentru a afla UUID-ul alocat unei partiti noi formatate

`lsblk` - pentru a vizualiza toate discurile din sistem - mai multe comenzi pt investigarea spatiului de stocare avem poze

`git init` → creează repo `git status` → verifică starea `git add` → pune fișiere în staging `git commit` → salvează versiune `git log` → istoric `git checkout -b` → branch nou `git merge` → combină branch-uri `git clone` → copiază repo `git push` → urcă pe server `git pull` → descarcă de pe server